



INTERVIEW REVISION SHEET

NODE.JS INTERVIEW REVISION SHEET

47+ must-know questions every backend developer should revise before a Node.js interview.

47+
QUESTIONS

8
TOPICS

21
SLIDES

INCLUDES

- ✓ Node.js Fundamentals
- ✓ Event Loop
- ✓ Async Programming
- ✓ Express & Middleware
- ✓ Streams & Buffers
- ✓ Authentication (JWT)
- ✓ Scaling & Clustering
- ✓ System Design Concepts
- ✓ Error Handling
- ✓ Testing & Debugging
- ✓ Security Basics
- ✓ Databases with Node.js

PERFECT FOR

- ✓ Backend Developers
- ✓ Node.js Interviews
- ✓ Quick Revision
- ✓ Placement Prep

LEARN • GROW • GO FURTHER

Save this sheet before your next interview.

NODE.JS FUNDAMENTALS

1 What is Node.js?

Node.js is a JavaScript runtime built on Chrome's V8 engine that lets you run JavaScript outside the browser, mainly on servers. It is event-driven and non-blocking, which makes it well suited for scalable network applications.

```
console.log("Hello from Node.js");  
// Run with: $ node app.js
```

2 Why is Node.js fast?

Node.js gets its speed from three things working together:

- The V8 engine compiles JavaScript straight to machine code
- Non-blocking I/O means it never waits idly on disk or network calls
- An event-driven architecture processes many requests concurrently on one thread

Instead of waiting for a slow task like a database query, Node.js keeps handling other requests and comes back once the result is ready.

3 What is the V8 engine?

V8 is Google's open-source JavaScript engine, also used in Chrome. It compiles JavaScript directly into optimized machine code instead of interpreting it line by line, which makes execution very fast.

```
let sum = 10 + 20;  
console.log(sum); // 30
```

4 Is Node.js single-threaded?

Yes — the main event loop runs on a single thread. However, heavy or blocking operations (file I/O, crypto, DNS lookups) are offloaded internally to a worker pool called libuv, so the main thread is never blocked.

5 Where is Node.js used?

Node.js is a popular choice for I/O-heavy backend systems.

- REST APIs and GraphQL servers
- Real-time apps such as chat and notifications
- Microservices
- Streaming services
- Server-side rendering (SSR)

6 What is npm?

npm (Node Package Manager) is the default package manager for Node.js. It is used to install, share and manage third-party libraries and project dependencies.

```
npm install express
npm install -g nodemon
```

7 What is package.json?

package.json is the manifest file of a Node.js project. It stores project metadata, dependencies, dev dependencies, npm scripts and version information.

```
{
  "name": "node-app",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

8 What is npx?

npx is a tool bundled with npm that runs a package's binary without installing it globally first. It always fetches the latest version unless one is cached.

```
npx create-react-app myapp
```

9 What are Node.js modules?

Modules are reusable, self-contained pieces of code that can be imported wherever needed. Node.js supports three kinds of modules:

- Core modules — built into Node.js itself
- Local modules — files you create in your own project
- Third-party modules — packages installed via npm

10 What are core modules?

Core modules ship with Node.js itself, so no installation is required. Some of the most commonly used ones:

- http — build web servers
- fs — read and write files
- path — work with file paths
- os — get operating-system information
- crypto — hashing and encryption
- events — build event emitters

```
const os = require("os");
console.log(os.platform());
```

11 What is the difference between CommonJS and ES Modules?

CommonJS (require/module.exports) is Node's original, synchronous module system. ES Modules (import/export) are the standard JavaScript module syntax, loaded asynchronously and supported natively since Node 12+ with the .mjs extension or "type": "module" in package.json.

```
// CommonJS
const fs = require("fs");
module.exports = fs;

// ES Module
import fs from "fs";
export default fs;
```

THE EVENT LOOP

12 What is the event loop?

The event loop is the mechanism that lets Node.js perform non-blocking I/O despite being single-threaded. It continuously checks the call stack and callback queues, executing callbacks whose asynchronous work has completed.

13 What are the phases of the event loop?

Each loop iteration ('tick') moves through six phases in order:

- Timers — runs setTimeout / setInterval callbacks
- Pending callbacks — deferred I/O callbacks
- Poll — retrieves new I/O events; executes I/O callbacks
- Check — runs setImmediate() callbacks
- Close callbacks — e.g. socket.on('close', ...)

Microtasks (Promises, process.nextTick) run between every phase, not as a separate phase.

14 What is the difference between process.nextTick() and setImmediate()?

process.nextTick() queues a callback to run immediately after the current operation, before the event loop continues — even before Promises resolve. setImmediate() queues a callback to run in the Check phase, after the current poll phase completes.

```
console.log("start");
setImmediate(() => console.log("immediate"));
process.nextTick(() => console.log("nextTick"));
console.log("end");
// start -> end -> nextTick -> immediate
```

15 What is the call stack in Node.js?

The call stack is a LIFO structure that tracks function calls currently being executed. When a function is invoked it's pushed onto the stack; when it returns, it's popped off. A long-running synchronous function that never returns will block the entire event loop.

16 What is the difference between microtasks and macrotasks?

Microtasks (Promise callbacks, process.nextTick, queueMicrotask) always run before the next macrotask, and the microtask queue is fully drained between phases. Macrotasks (setTimeout, setInterval, setImmediate, I/O) are scheduled to run in a later event-loop phase.

ASYNCHRONOUS PROGRAMMING

17 What is a callback function?

A callback is a function passed as an argument to another function, to be invoked once an asynchronous operation completes.

```
fs.readFile("file.txt", (err, data) => {
  if (err) return console.error(err);
  console.log(data.toString());
});
```

18 What is callback hell, and how do you avoid it?

Callback hell is deeply nested, hard-to-read callbacks that occur when multiple asynchronous operations depend on each other. It is avoided by:

- Using Promises instead of nested callbacks
- Using `async/await` for a synchronous-looking flow
- Breaking logic into small, named functions

19 What is a Promise?

A Promise is an object representing the eventual completion (or failure) of an asynchronous operation. It has three states: pending, fulfilled, and rejected.

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve("done"), 1000);
});
p.then(res => console.log(res)).catch(err => console.error(err));
```

20 What are `async/await`?

`async/await` is syntactic sugar over Promises that lets asynchronous code be written and read like synchronous code. An `async` function always returns a Promise; `await` pauses execution until that Promise settles.

```
async function getUser() {
  try {
    const res = await fetch("/api/user");
    return await res.json();
  } catch (err) {
    console.error(err);
  }
}
```

21 What does `Promise.all()` do, and how is it different from `Promise.allSettled()`?

`Promise.all()` runs promises in parallel and resolves once every one succeeds — but it rejects immediately if any single promise fails. `Promise.allSettled()` also runs them in parallel, but always resolves after all promises finish, returning the status ('fulfilled' or 'rejected') of each one.

```
const results = await Promise.allSettled([p1, p2, p3]);
```

22 What is the difference between synchronous and asynchronous code in Node.js?

Synchronous code executes line by line, blocking further execution until the current line finishes. Asynchronous code lets Node.js start an operation (like a file read or network call) and continue running other code while it waits, resuming via a callback, Promise, or `async/await` once it's done.

EXPRESS & MIDDLEWARE

23 What is Express.js?

Express is a minimal, unopinionated web framework for Node.js. It simplifies routing, middleware handling, and building REST APIs on top of Node's built-in http module.

```
const express = require("express");
const app = express();
app.get("/", (req, res) => res.send("Hello"));
app.listen(3000);
```

24 What is middleware in Express?

Middleware are functions that run during the request-response cycle, with access to req, res, and the next() function. They can modify the request, end the response, or pass control to the next middleware.

```
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});
```

25 What is the difference between app.use() and app.get()?

app.use() mounts middleware for all HTTP methods on a path (and any sub-paths). app.get() registers a route handler that only responds to GET requests on an exact path.

26 How do you handle errors in Express?

Express uses special error-handling middleware that takes four arguments (err, req, res, next). It should be registered last, after all other routes and middleware.

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: "Something went wrong" });
});
```

27 What is CORS, and how do you enable it in Express?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts web pages from making requests to a different origin than the one that served them. In Express it's commonly enabled with the cors package.

```
const cors = require("cors");
app.use(cors());
```

28 What is the purpose of the next() function?

next() hands control from the current middleware to the next one in the stack. If it's not called (and the response isn't sent), the request will hang. Calling next(err) skips to the nearest error-handling middleware.

STREAMS & BUFFERS

29 What are streams in Node.js?

Streams are objects that let you read or write data piece by piece (in chunks) instead of loading it all into memory at once — ideal for large files or network data.

30 What are the different types of streams?

Node.js has four fundamental stream types:

- Readable — data can be read from it (e.g. fs.createReadStream)
- Writable — data can be written to it (e.g. fs.createWriteStream)
- Duplex — both readable and writable (e.g. a TCP socket)
- Transform — a duplex stream that can modify data as it passes through (e.g. zlib.createGzip)

31 What is a Buffer?

A Buffer is a fixed-size chunk of raw binary data stored outside the V8 heap. Node.js uses Buffers to handle binary data such as file contents, TCP streams and images.

```
const buf = Buffer.from("Hello");  
console.log(buf); //
```

32 What is backpressure in streams?

Backpressure happens when a writable stream can't process incoming data as fast as a readable stream is producing it. write() returns false when its internal buffer is full, signalling the source to pause until a 'drain' event fires.

33 How do you pipe streams together?

The .pipe() method connects a readable stream's output directly to a writable stream's input, automatically managing backpressure for you.

```
fs.createReadStream("input.txt")  
  .pipe(zlib.createGzip())  
  .pipe(fs.createWriteStream("input.txt.gz"));
```

AUTHENTICATION (JWT) & SECURITY

34 What is JWT, and how does it work?

A JSON Web Token (JWT) is a compact, self-contained token used to securely transmit claims between two parties. It has three parts — header, payload, and signature — and is signed (not encrypted) so the server can verify it hasn't been tampered with.

```
const jwt = require("jsonwebtoken");
const token = jwt.sign({ id: user.id }, SECRET, { expiresIn: "1h" });
```

35 How do you implement authentication using JWT in Node.js?

A typical flow: the user logs in with credentials, the server verifies them and signs a JWT, the client stores it and sends it in the Authorization header on future requests, and middleware verifies the token before granting access.

```
function auth(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1];
  try {
    req.user = jwt.verify(token, SECRET);
    next();
  } catch {
    res.status(401).json({ message: "Invalid token" });
  }
}
```

36 What is the difference between authentication and authorization?

Authentication verifies who a user is (login with credentials). Authorization determines what an authenticated user is allowed to do (permissions and roles).

37 How do you securely store passwords in Node.js?

Never store plain-text passwords. Hash them with a slow, salted algorithm such as bcrypt before saving, and compare hashes on login.

```
const bcrypt = require("bcrypt");
const hash = await bcrypt.hash(password, 10);
const valid = await bcrypt.compare(password, hash);
```

38 What is Helmet.js used for?

Helmet is Express middleware that sets a collection of security-related HTTP headers (like Content-Security-Policy and X-Frame-Options) to protect against common attacks such as clickjacking and XSS.

```
const helmet = require("helmet");
app.use(helmet());
```

39 What is rate limiting, and why is it important?

Rate limiting caps how many requests a client can make in a given time window. It protects APIs from abuse, brute-force attacks and accidental overload, and is commonly implemented with express-rate-limit or a Redis-backed counter.

SCALING & CLUSTERING

40 What is the cluster module in Node.js?

The cluster module lets a Node.js application spawn multiple worker processes — one per CPU core — that share the same server port, allowing it to take advantage of multi-core systems since a single Node process runs on only one core.

```
const cluster = require("cluster");
const os = require("os");
if (cluster.isPrimary) {
  os.cpus().forEach(() => cluster.fork());
} else {
  require("./server");
}
```

41 What is the difference between clustering and worker threads?

Clustering spawns separate processes, each with its own memory and event loop, ideal for scaling network servers across CPU cores. Worker threads (the worker_threads module) run in the same process and can share memory, better suited for CPU-intensive tasks like image processing or data crunching.

42 What is a load balancer, and how does it help Node.js apps scale?

A load balancer sits in front of multiple server instances and distributes incoming traffic across them (round-robin, least-connections, etc.). It enables horizontal scaling, improves availability, and prevents any single instance from being overwhelmed.

43 What is horizontal vs vertical scaling?

Vertical scaling means adding more resources (CPU, RAM) to a single machine. Horizontal scaling means adding more machines or instances and distributing load between them — the more common approach for Node.js apps because it also improves fault tolerance.

SYSTEM DESIGN & DATABASES

44 How would you design a rate limiter for an API?

A common approach uses the token bucket or sliding-window-counter algorithm backed by a fast shared store like Redis, so limits apply consistently across multiple server instances.

- Store a request counter per client/IP with an expiry window in Redis
- Increment on each request; reject once the count exceeds the limit
- Return standard 429 Too Many Requests with a Retry-After header

45 What is the difference between SQL and NoSQL databases?

SQL databases (PostgreSQL, MySQL) are relational, with fixed schemas and strong consistency — great for structured, transactional data. NoSQL databases (MongoDB, Redis) are more flexible and horizontally scalable — good for unstructured or fast-changing data.

```
// SQL (via pg)
const { rows } = await pool.query("SELECT * FROM users");

// NoSQL (via mongoose)
const users = await User.find();
```

46 What is caching, and how would you use Redis in a Node.js system?

Caching stores frequently accessed data in fast, in-memory storage to reduce load on the primary database and speed up responses. Redis is a popular in-memory store used for caching, session storage, and rate limiting in Node.js systems.

```
const client = redis.createClient();
await client.set("user:1", JSON.stringify(user), { EX: 3600 });
const cached = await client.get("user:1");
```

47 How do you handle uncaught exceptions and unhandled promise rejections in production?

Listen for both process-level events, log the error for diagnostics, and gracefully shut the process down so a process manager (PM2, Kubernetes) can restart it in a clean state — never try to keep running after an uncaught exception.

```
process.on("uncaughtException", (err) => {
  console.error(err);
  process.exit(1);
});
process.on("unhandledRejection", (reason) => {
  console.error(reason);
  process.exit(1);
});
```



Keep Learning with **Skilligo**

You've just revised 47+ core Node.js interview questions spanning fundamentals, the event loop, async programming, Express, streams, security, scaling and system design. Bookmark this sheet and revisit it before every interview.

LEARN • GROW • GO FURTHER

FOLLOW SKILLIGO